

F1

Knowledge System

Web Semantics — Practical Assignment 2 — Report

Authors	Rodrigo Abreu (113626), Hugo Ribeiro (113402)
Course	Web Semantics
Year	2025–2026
Technology	Django 6 · GraphDB · OWL 2 DL · SPIN · RDFa · MF2

June 2, 2026

Contents

1	Introduction	2
1.1	Technology Stack	3
1.2	Dataset	3
2	Ontology Definition (RDFS and OWL)	3
2.1	OWL Profile	3
2.2	Class Hierarchy	4
2.3	Base Class Inference Without Explicit <code>rdf:type</code>	4
2.3.1	<code>rdfs:domain</code> (for GraphDB)	4
2.3.2	<code>owl:equivalentClass</code> (for Protégé / HermiT)	5
2.4	Why <code>rdfs:domain</code> is Omitted on Foreign-Key Properties	5
2.5	Design Decision: Inferred vs. Asserted <code>rdf:type</code>	5
2.6	Key OWL Axioms	6
2.7	Restriction Axioms for Inferred Subclasses	7
2.8	Inference Verification	7
2.9	Protégé Validation with HermiT Reasoner	7
2.9.1	OWL 2 Datatype Map Compliance	7
2.9.2	Design Decision	8
2.9.3	HermiT Verification Results	8
3	Quantitative Analysis	9
3.1	Ontology Metrics	10
3.2	Complete Class Inventory	12
3.3	Complete Object Property Inventory	13
3.4	Ontology Visualizations	14
3.5	Complete SPIN Rule Inventory	18
3.6	GraphDB Validation	18
4	Inference Rules (SPIN)	19
4.1	Championship Inference	20
4.2	Teammate Inference	21
4.3	Classification Rules	21
4.4	Podium Decomposition Rules	22
4.5	Pole Position and Race Achievement Rules	22
4.6	RDF Reification	23
5	New Operations on the Data (SPARQL)	23
5.1	World Champions Who Raced as Teammates	24
5.2	Constructors That Fielded the Most World Champions	24
5.3	SPARQL Property Paths via <code>rdfs:subPropertyOf</code>	24
5.4	Reification Query: Championship Final Points	24
5.5	Veteran World Champions (Class Intersection)	25
5.6	Two-Hop Teammate Graph Traversal	25
6	Use and Integration of Wikidata and DBpedia	25
6.1	Driver Enrichment (Wikidata)	26

6.2	Constructor Enrichment (Wikidata)	26
6.3	Circuit Enrichment (DBpedia)	27
6.4	Race Enrichment (DBpedia)	28
7	Publication of Semantic Data (RDFa and Microformats)	28
7.1	RDFa 1.1 Lite	28
7.2	Microformats 2	29
7.3	Why Both Standards?	30
7.4	Microformats 2 Parser Demonstration	30
7.5	Validation of Published Semantic Data	30
7.5.1	RDFa 1.1 Validation Results	31
7.5.2	Microformats 2 Validation Results	32
8	Application Functionalities (UI)	33
8.1	Pages Driven by Inferred Knowledge	33
8.1.1	Hall of Fame Dashboard (/hall-of-fame/)	33
8.1.2	Pole Position Leaders (/p1/)	33
8.1.3	Hat Trick Hall of Fame (/hat-tricks/)	34
8.1.4	Microformats 2 Parser Demonstration	34
8.2	Hero Section Enrichment	35
8.3	Navigation Structure	35
8.4	Card Hover Expansion	36
8.5	Deep Links	36
8.6	Podium Visibility Control	36
9	Conclusions	36
9.1	Key Achievements	36
9.2	Challenges and Solutions	37
10	Configuration to Run the Application	37
10.1	Prerequisites	37
10.2	Quick Start	37
10.3	Manual Step-by-Step	38
10.4	Protégé Validation	39
10.5	Re-running Inference	39
10.6	Data Files Summary	39

1. Introduction

This report covers **Practical Assignment 2** (TP2), which extends the F1 Knowledge Graph Explorer developed in TP1. The first assignment established the foundation — a Django application backed by GraphDB with all domain data stored as RDF triples and queried through SPARQL — but without any formal ontology or reasoning capabilities. TP2 adds a complete semantic knowledge layer on top of that foundation, enabling entity classification, materialisation of derived facts, semantic annotation of HTML pages, and live integration with external linked data sources.

The key contributions of TP2 are:

- An **OWL 2 DL ontology** that formally describes the F1 domain, enabling automatic entity classification directly from the data properties of each entity — without any explicit `rdf:type` assertions in the facts file.
- **21 SPIN rules** (SPARQL INSERT WHERE) that materialise inferences OWL cannot express: World Champions, Veterans, Historic Circuits, podium decomposition (1st/2nd/3rd), pole positions, hat tricks, and the teammate relation between drivers who raced together.
- **RDF reification** annotating every championship-winning triple with the driver's final points total.
- **Live semantic enrichment** from Wikidata and DBpedia via SPARQLWrapper: driver death dates, team logos, circuit lap records, and more.
- **RDFa 1.1 Lite** (Schema.org) and **Microformats 2** embedded in every detail page, making the site machine-readable by search engines and IndieWeb tools alike.
- New application pages driven entirely by inferred knowledge: World Champions, Multi-Champions, Veterans, Constructor Champions, Pole Position Leaders, Hat Trick Hall of Fame, and a Hall of Fame dashboard.

1.1. Technology Stack

Technology	Role	Where used
Python 3.12 / Django 5	Web framework and data pipeline	All views, management commands
GraphDB 10 (Ontotext)	RDF triple store with OWL-Max reasoner	Primary knowledge base
SPARQL 1.1	Knowledge retrieval and SPIN rules	21 rules; all Django views
RDF / N-Triples	Data serialisation	<code>formula1_integrated.nt</code>
RDFS	Schema and subclass hierarchy	Ontology (domain, range, subClassOf)
OWL 2 DL	Ontology axioms and offline reasoning	<code>formula1_ontology.ttl</code>
Protégé + HermiT 1.4.3	Ontology editing and consistency check	Offline validation
SPIN (via SPARQL UPDATE)	Materialised inference beyond OWL	<code>spin_rules.py</code> (21 rules)
SPARQLWrapper 2	Federated queries to Wikidata / DBpedia	<code>services/external_data.py</code>
RDFa 1.1 Lite	Semantic HTML annotation (Schema.org)	6 detail page templates
Microformats 2 mf2py	Structured data in HTML Server-side MF2 parsing	6 detail page templates <code>/api/parse-microformats/</code>

1.2. Dataset

We used the **Kaggle Formula 1 World Championship** dataset (1950–2024), which follows the Ergast-style CSV schema. It covers 75 seasons, 861 drivers, 212 constructors, 77 circuits, and approximately 25,000 race results distributed across 13 CSV files. These are loaded into a GraphDB repository named `ws-formula1-owlmax`.

2. Ontology Definition (RDFS and OWL)

We designed the ontology as a single Turtle file (`data/rdf/formula1_ontology.ttl`), following the OWL 2 DL profile. It is the **only hand-authored source file** in the data pipeline — every other RDF file is either generated by a script or produced by the reasoner. The ontology cannot itself be generated by a script because it encodes design decisions (class hierarchies, axioms, property chains) that require human authorship. However, it is fully reproducible: the file is version-controlled in git, and `scripts/merge_integrated.py` packages it with the facts file every time a reload is needed.

2.1. OWL Profile

We chose **OWL 2 DL** because it is a decidable fragment of first-order logic with well-defined reasoning guarantees, and because GraphDB supports the **OWL-Max with RDFS** ruleset out of the box. This ruleset applies `rdfs:domain`, `rdfs:range`, `rdfs:subClassOf`,

`owl:inverseOf`, and `owl:SymmetricProperty` entailments at triple load time — meaning the reasoner fires before any SPARQL query runs, and the inferred triples are already in the graph when the application queries it.

2.2. Class Hierarchy

We defined 20 asserted OWL classes (plus 7 SPIN-inferred subclasses added at runtime) organised into a domain-aligned hierarchy. A complete table of all 27 classes with their sources is provided in Section 3. The key structural excerpt is below:

```
f1:Entity      rdfs:subClassOf owl:Thing .
f1:Person      rdfs:subClassOf f1:Entity .
f1:Team        rdfs:subClassOf f1:Entity .
f1:Venue       rdfs:subClassOf f1:Entity .
f1:Driver      rdfs:subClassOf f1:Person .
f1:Constructor rdfs:subClassOf f1:Team .
f1:Circuit     rdfs:subClassOf f1:Venue .
f1:Race        rdfs:subClassOf f1:Event .
f1:Season      rdfs:subClassOf f1:Event .
f1:Result      rdfs:subClassOf f1:PerformanceRecord .
f1:WorldChampion rdfs:subClassOf f1:Driver .
f1:MultichampionDriver rdfs:subClassOf f1:WorldChampion .
f1:Veteran     rdfs:subClassOf f1:Driver .
f1:ConstructorChampion rdfs:subClassOf f1:Constructor .
f1:ActiveCircuit rdfs:subClassOf f1:Circuit .
f1:HistoricCircuit rdfs:subClassOf f1:Circuit .
f1:PodiumResult rdfs:subClassOf f1:Result .
```

Listing 1: Top-level class hierarchy in the ontology

2.3. Base Class Inference Without Explicit `rdf:type`

A central design requirement was that the facts file should contain **no explicit** `rdf:type` `f1:Driver` (or `Constructor`, `Circuit`) assertions. The ontology must infer these types purely from the properties present on each entity.

This is achieved using two complementary mechanisms:

2.3.1. *rdfs:domain* (for *GraphDB*)

Each “signature” property that appears exclusively on one entity class is declared with `rdfs:domain`. GraphDB’s OWL-Max ruleset then infers `rdf:type` at load time from any occurrence of that property:

```
f1:driverRef    rdfs:domain f1:Driver .
f1:constructorRef rdfs:domain f1:Constructor .
f1:circuitRef   rdfs:domain f1:Circuit .
```

Listing 2: `rdfs:domain` declarations as inference anchors

So `<driver/1> f1:driverRef "hamilton"` causes GraphDB to automatically infer `<driver/1> rdf:type f1:Driver` without any explicit assertion.

2.3.2. *owl:equivalentClass* (for Protégé / HermiT)

For validation in Protégé, the OWL reasoner HermiT requires a *necessary and sufficient* condition. Each base class is therefore defined with `owl:equivalentClass` using a `someValuesFrom` restriction:

```
f1:Driver  owl:equivalentClass  [
    rdf:type          owl:Restriction ;
    owl:onProperty   f1:driverRef ;
    owl:someValuesFrom xsd:string
] .
```

Listing 3: Bidirectional `equivalentClass` definition for Driver

This means: an individual is a `f1:Driver` **if and only if** it has at least one `f1:driverRef` value. HermiT can classify anonymous individuals without an explicit type assertion, making the ontology verifiable in Protégé.

Verification: Loading the facts file alone into GraphDB (without the ontology) returns 0 results for `SELECT * WHERE {?d rdf:type f1:Driver}`. After loading the integrated file (ontology + facts), the same query returns 861 drivers. The ontology, not the data, does the classification.

2.4. Why `rdfs:domain` is Omitted on Foreign-Key Properties

Properties like `f1:driverId`, `f1:constructorId`, and `f1:raceId` appear as **literal columns on multiple entity types**. For example, a `Result` row carries both a `f1:driverId` and a `f1:constructorId` literal. Declaring `rdfs:domain f1:Driver` on `f1:driverId` would cause GraphDB to infer that every `Result` individual is also a `Driver`, immediately triggering the `owl:disjointWith f1:Constructor` inconsistency. Domain declarations are intentionally omitted for these FK properties.

2.5. Design Decision: Inferred vs. Asserted `rdf:type`

A deliberate asymmetry in the data pipeline distinguishes between entities whose type can be *inferred* by the reasoner and those that require *explicit* `rdf:type` assertions in the facts file.

Entities with inferred type (no explicit `rdf:type` in the facts file): `f1:Driver`, `f1:Constructor`, `f1:Circuit`. Each of these has a *unique signature property* that appears exclusively on that entity class and on no other:

- `f1:driverRef` $\Rightarrow$ only Driver entities carry this property
- `f1:constructorRef` $\Rightarrow$ only Constructor entities carry this property
- `f1:circuitRef` $\Rightarrow$ only Circuit entities carry this property

Declaring `rdfs:domain f1:Driver` on `f1:driverRef` is therefore safe: no other entity type in the dataset ever carries a `driverRef` literal. GraphDB's OWL-Max ruleset fires the inference at load time, meaning `rdf:type f1:Driver` never needs to appear in the facts file.

Entities with explicit `rdf:type` (asserted in the facts file): `f1:Race`, `f1:Season`, `f1:Result`, `f1:DriverStanding`, `f1:QualifyingResult`, etc. These entities share many properties with other types. For example, both `Race` and `Season` carry `f1:year`; both `Result` and `DriverStanding` carry `f1:driver` and `f1:points`. There is no safe signature property that would let the reasoner distinguish them without risk of misclassification.

Consequence for SPIN rules. Because some entities share properties, SPIN rules that pattern-match on shared properties must include explicit type guards to avoid cross-type false positives. The `infer_wonChampionship` rule, for instance, uses:

```
?season rdf:type f1:Season . -- type guard
```

Without this guard, the `?season f1:year ?yr` pattern would also match `Race` entities (which also carry `f1:year`), producing 1,200 triples instead of the correct 75. The explicit `rdf:type` assertions for `Season` and `Race` are therefore a deliberate design requirement, not an oversight.

Summary: the principle applied throughout the ontology is *use inference where a unique exclusive signature property exists; use explicit `rdf:type` everywhere else*. This gives the most concise facts file while keeping SPARQL patterns correct and avoiding ontology inconsistencies.

2.6. Key OWL Axioms

Construct	Applied to	Purpose
<code>owl:FunctionalProperty</code>	<code>race</code> , <code>driver</code> , <code>status</code> , <code>circuit</code> , <code>season</code> , <code>heldAt</code>	At most one value per subject.
<code>owl:SymmetricProperty</code>	<code>wasTeammate</code>	$x \text{ wasTeammate } y \Rightarrow y \text{ wasTeammate } x$; GraphDB materialises the inverse.
<code>owl:inverseOf</code>	<code>hadDriver</code> $\leftrightarrow$ <code>drovFor</code>	Constructor $\rightarrow$ Driver navigation without storing the triple twice.
<code>rdfs:subPropertyOf</code>	<code>wonRace</code> $\sqsubseteq$ <code>achievedPodium</code> $\sqsubseteq$ <code>finishedSecond</code> $\sqsubseteq$ <code>achievedPodium</code> $\sqsubseteq$ <code>competedIn</code>	<code>Race</code> wins and podium finishes also appear as <code>competedIn</code> triples; enables property-path queries.
<code>owl:maxCardinality 1</code>	<code>Race</code> $\sqsubseteq \leq 1$ <code>circuit</code>	Each race is held at exactly one circuit.
<code>owl:disjointWith</code>	<code>Driver/Constructor</code> , <code>Driver/Circuit</code> , <code>Race/Season</code> , <code>Re-</code> <code>sult/QualifyingResult</code> , ...	Detects modelling errors during Protégé validation (7 pairs total).

Complete tables of all 27 classes and 21 object properties are provided in Section 3 (*Quantitative Analysis*).

2.7. Restriction Axioms for Inferred Subclasses

Three inferred subclasses carry `rdfs:subClassOf` restriction axioms that document the structural intent — even though the actual classification is performed by SPIN rules (Section 4):

```
f1:WorldChampion  rdfs:subClassOf  [
    rdf:type          owl:Restriction ;
    owl:onProperty  f1:wonChampionship ;
    owl:someValuesFrom f1:Season
] .
```

Listing 4: Restriction on WorldChampion: must have at least one wonChampionship

2.8. Inference Verification

After loading the integrated file (389 ontology triples + 6.6M fact triples) and running all 21 SPIN rules, all inference checks pass:

Mechanism	Inferred	Count
<code>rdfs:domain driverRef</code>	<code>f1:Driver</code> entities	861
<code>rdfs:domain constructorRef</code>	<code>f1:Constructor</code> entities	212
<code>rdfs:domain circuitRef</code>	<code>f1:Circuit</code> entities	77
SPIN <code>classify_WorldChampion</code>	<code>f1:WorldChampion</code>	34
SPIN <code>classify_MultichampionDriver</code>	<code>f1:MultichampionDriver</code>	17
SPIN <code>classify_Veteran</code>	<code>f1:Veteran</code>	86
SPIN <code>classify_ConstructorChampion</code>	<code>f1:ConstructorChampion</code>	17
SPIN <code>classify_ActiveCircuit</code>	<code>f1:ActiveCircuit</code>	24
SPIN <code>classify_HistoricCircuit</code>	<code>f1:HistoricCircuit</code>	45
SPIN <code>classify_PodiumResult</code>	<code>f1:PodiumResult</code>	3,397
SPIN <code>infer_wonRace</code>	<code>f1:wonRace</code> triples	1,128
SPIN <code>infer_startedFromP1</code>	<code>f1:startedFromP1</code> triples	1,135
SPIN <code>infer_convertedP1ToWin</code>	<code>f1:convertedP1ToWin</code> triples	486
SPIN <code>infer_setFastestLap</code>	<code>f1:setFastestLap</code> triples	410
SPIN <code>infer_achievedHatTrick</code>	<code>f1:achievedHatTrick</code> triples	69
SPIN <code>infer_wasTeammate</code>	<code>f1:wasTeammate</code> triples	10,012
SPIN <code>infer_wonChampionship</code>	<code>f1:wonChampionship</code> triples	75
<code>owl:inverseOf</code>	<code>f1:hadDriver</code> triples	2,150
SPIN <code>infer_heldAt</code>	<code>f1:heldAt</code> triples	1,125

2.9. Protégé Validation with HermiT Reasoner

The ontology was formally validated using the **HermiT 1.4.3** DL reasoner (bundled with Protégé 5.6.9) running programmatically via the `owlready2` Python library. This section documents the complete analysis, including a real issue encountered and the design decision taken to resolve it.

2.9.1. OWL 2 Datatype Map Compliance

During validation, HermiT raised an `UnsupportedDatatypeException` for three `rdfs:range` declarations:

```

f1:dob      rdfs:range xsd:date .    -- NOT in OWL 2 datatype map
f1:date     rdfs:range xsd:date .    -- NOT in OWL 2 datatype map
f1:year     rdfs:range xsd:gYear .   -- NOT in OWL 2 datatype map

```

Listing 5: Original range declarations causing HermiT rejection

The OWL 2 specification defines a strict *datatype map* — a subset of XML Schema datatypes that OWL 2 DL reasoners are required to support. This map includes `xsd:dateTime` and `xsd:dateTimeStamp` but **not** `xsd:date` or `xsd:gYear`. The teacher’s slides state that “A OWL faz uso dos tipos de dados definidos no XML Schema” (OWL uses XML Schema datatypes), but the key constraint is that only a *subset* of XML Schema types are in the OWL 2 DL datatype map.

2.9.2. Design Decision

The `rdfs:range` declaration is defined in RDFS as *optional*: it states what kind of values a property *may* take, but its absence does not prevent the property from functioning. The decision was to **remove the three non-compliant range declarations** rather than replacing them with an incorrect type (e.g., `xsd:dateTime`) or a loss of type information (e.g., `xsd:string`).

Justification: The semantic power of the ontology is entirely unchanged. All inference mechanisms — `rdfs:domain` for base class classification, `owl:equivalentClass` for Driver/Constructor/Circuit, `owl:disjointWith` for consistency checking, `owl:FunctionalProperty`, `owl:SymmetricProperty`, `owl:inverseOf`, and `rdfs:subPropertyOf` — are independent of these three range declarations. The domain declarations (`f1:dob rdfs:domain f1:Driver`, `f1:year rdfs:domain f1:Event`, `f1:date rdfs:domain f1:Race`) are preserved and still provide useful type inference. The actual data values in GraphDB remain typed as `xsd:date` and `xsd:gYear`, which GraphDB handles correctly under its OWL-Max ruleset.

2.9.3. HermiT Verification Results

HermiT was run twice: once on the original ontology (838 ms), and again after adding the two new properties (`f1:finishedSecond`, `f1:finishedThird`) and restructuring the `subPropertyOf` chain (`wonRace  $\sqsubseteq$  achievedPodium`). Both runs produced identical consistency results. The updated ontology (389 triples, 27 classes, 21 object properties) completed reasoning in **1,333 ms**.

Check	Result
Consistency	CONSISTENT — no exception, no contradiction
Unsatisfiable classes	0 — every named class has a non-empty extension
Equivalent class definitions	Driver $\equiv \exists \text{driverRef.string}$, Constructor $\equiv \exists \text{constructorRef.string}$, Circuit $\equiv \exists \text{circuitRef.string}$
Property hierarchy	wonRace $\sqsubseteq$ achievedPodium $\sqsubseteq$ competedIn, finishedSecond $\sqsubseteq$ achievedPodium, finishedThird $\sqsubseteq$ achievedPodium
Functional properties	circuit, driver, heldAt, race, season, status
Symmetric properties	wasTeammate
Disjoint classes	7 pairs verified (Driver $\perp$ Constructor, Driver $\perp$ Circuit, Circuit $\perp$ Constructor, Race $\perp$ Season, Driver $\perp$ Race, Result $\perp$ QualifyingResult, LapTime $\perp$ Result)

The full inferred class hierarchy produced by Hermit:

Listing 6: Class hierarchy inferred by Hermit from owl:equivalentClass + rdfs:subClassOf axioms

```

owl:Thing
  Entity
    Person  -> Driver  -> WorldChampion  -> MultichampionDriver
              -> Veteran
    Team    -> Constructor  -> ConstructorChampion
    Venue   -> Circuit      -> ActiveCircuit
              -> HistoricCircuit

  Event
    Race
    Season
  PerformanceRecord
    Result  -> PodiumResult
    QualifyingResult
    LapTime
    PitStop
    ConstructorResult
  Standing
    DriverStanding
    ConstructorStanding
  Status

```

This hierarchy proves the ontology’s inference power: the `WorldChampion`, `Veteran`, `ConstructorChampion`, `ActiveCircuit`, `HistoricCircuit`, and `PodiumResult` subclasses are all correctly positioned by Hermit using `rdfs:subClassOf` alone, confirming that the ontology structure is semantically sound.

3. Quantitative Analysis

This section provides objective, verifiable evidence of the semantic contribution made by the ontology and inference system.

3.1. Ontology Metrics

Ontology metric	Count	Notes
OWL classes (total)	27	7 abstract + 13 concrete + 7 SPIN-inferred
Concrete asserted classes	13	Populated from CSV
SPIN-inferred subclasses	7	Not in facts file
Object properties	21	6 raw + 15 SPIN-materialised
Datatype properties	35	Covering all CSV columns
owl:FunctionalProperty	6	race, driver, circuit, status, season, heldAt
owl:SymmetricProperty	1	f1:wasTeammate
owl:inverseOf pairs	1	hadDriver / drovFor
rdfs:subPropertyOf chains	6	All under f1:competedIn
owl:disjointWith pairs	7	Prevents cross-type inference
owl:maxCardinality restrictions	1	Race→circuit (max 1)
owl:equivalentClass (Protégé)	3	Driver, Constructor, Circuit base inference
owl:someValuesFrom restrictions	2	WorldChampion, ConstructorChampion
HermiT reasoning time	1,333 ms	owlready2 on Python 3.12 (21 properties)
Unsatisfiable classes (HermiT)	0	Ontology is consistent

Data pipeline metric	Count	Notes
Raw CSV files	13	Ergast F1 1950–2024
N-Triple facts	6,627,045	formula1.nt
Ontology triples	375	formula1_ontology.ttl
Integrated file triples	6,627,420	formula1_integrated.nt
OWL-Max inferred (at import)	≈ 1,950	rdfs:domain → rdf:type
SPIN-generated triples	22,921	21 rules (verified live, June 2026)
Total triples in repository	6,650,355	post-inference (verified live)
Drivers	861	OWL-Max inferred
Constructors	212	OWL-Max inferred
Circuits	77	OWL-Max inferred
Seasons	75	Explicit rdf:type
Races	1,125	Explicit rdf:type
Results	≈ 25,000	Explicit rdf:type

Page	RDFa type of	MF2 classes	Properties
Driver detail	schema:Person	h-card	name, nationality, birthDate, sameAs
Constructor detail	schema:SportsOrganization	h-card	name, nationality, sameAs
Circuit detail	schema:SportsActivityLocation	h-card, h-adr	name, address- Locality, address- Country, lat, lng
Race detail	schema:SportsEvent	h-event	name, location, startDate, sameAs
Season detail	schema:SportsEvent	h-event	name, startDate, sameAs
Champions list	schema:ItemList	—	itemListElement (Person $\times$ 34)
Homepage	schema:Dataset	—	name, description, license, temporal- Coverage

3.2. Complete Class Inventory

Class	Superclass	Type	Source
f1:Entity	—	Abstract root	Hand-authored
f1:Person	f1:Entity	Abstract	Hand-authored
f1:Team	f1:Entity	Abstract	Hand-authored
f1:Venue	f1:Entity	Abstract	Hand-authored
f1:Event	—	Abstract	Hand-authored
f1:PerformanceRecord	—	Abstract	Hand-authored
f1:Standing	—	Abstract	Hand-authored
f1:Driver	f1:Person	Concrete; asserted	OWL-Max (rdfs:domain)
f1:Constructor	f1:Team	Concrete; asserted	OWL-Max (rdfs:domain)
f1:Circuit	f1:Venue	Concrete; asserted	OWL-Max (rdfs:domain)
f1:Season	f1:Event	Concrete; asserted	Explicit in facts
f1:Race	f1:Event	Concrete; asserted	Explicit in facts
f1:Result	f1:PerformanceRecord	Concrete	Explicit in facts
f1:QualifyingResult	f1:PerformanceRecord	Concrete	Explicit in facts
f1:LapTime	f1:PerformanceRecord	Concrete	Explicit in facts
f1:PitStop	f1:PerformanceRecord	Concrete	Explicit in facts
f1:ConstructorResult	f1:PerformanceRecord	Concrete	Explicit in facts
f1:DriverStanding	f1:Standing	Concrete; asserted	Explicit in facts
f1:ConstructorStanding	f1:Standing	Concrete; asserted	Explicit in facts
f1:Status	—	Concrete; asserted	Explicit in facts
f1:WorldChampion	f1:Driver	SPIN-inferred	34 instances
f1:MultichampionDriver	f1:WorldChampion	SPIN-inferred	17 instances
f1:Veteran	f1:Driver	SPIN-inferred	86 instances
f1:ActiveCircuit	f1:Circuit	SPIN-inferred	24 instances
f1:HistoricCircuit	f1:Circuit	SPIN-inferred	45 instances
f1:ConstructorChampion	f1:Constructor	SPIN-inferred	17 instances
f1:PodiumResult	f1:Result	SPIN-inferred	3,397 instances

3.3. Complete Object Property Inventory

Property	Domain	Range	OWL axiom	Source
f1:race	(f1:Result)	f1:Race	Functional	Asserted
f1:driver	(f1:Result)	f1:Driver	Functional	Asserted
f1:constructor	(f1:Result)	f1:Constructor	—	Asserted
f1:status	f1:Result	f1:Status	Functional	Asserted
f1:circuit	f1:Race	f1:Circuit	Functional	Asserted
f1:season	f1:Race	f1:Season	Functional	Asserted
f1:heldAt	f1:Race	f1:Circuit	Functional	SPIN: infer_heldAt
f1:drovFor	f1:Driver	f1:Constructor	—	SPIN: infer_drovFor
f1:hadDriver	f1:Constructor	f1:Driver	inverseOf	OWL: inverseOf drovFor
f1:wasTeammate	f1:Driver	f1:Driver	Symmetric	SPIN: infer_wasTeammate
f1:competedIn	f1:Driver	f1:Race	—	SPIN (base)
f1:achievedPodium	f1:Driver	f1:Race	$\sqsubseteq$ competedIn	SPIN
f1:finishedSecond	f1:Driver	f1:Race	$\sqsubseteq$ achievedPodium	SPIN
f1:finishedThird	f1:Driver	f1:Race	$\sqsubseteq$ achievedPodium	SPIN
f1:wonRace	f1:Driver	f1:Race	$\sqsubseteq$ achievedPodium	SPIN (1,128)
f1:startedFromP1	f1:Driver	f1:Race	$\sqsubseteq$ competedIn	SPIN (1,135)
f1:setFastestLap	f1:Driver	f1:Race	$\sqsubseteq$ competedIn	SPIN (410)
f1:convertedP1ToWin	f1:Driver	f1:Race	$\sqsubseteq$ wonRace	SPIN (486)
f1:achievedHatTrick	f1:Driver	f1:Race	$\sqsubseteq$ wonRace	SPIN (69)
f1:wonChampionship	f1:Driver	f1:Season	—	SPIN (75)
f1:wonCtorChamp.	f1:Constructor	f1:Season	—	SPIN (17)
f1:finalPoints	rdf:Statement	xsd:decimal	—	Reification metadata

3.4. Ontology Visualizations

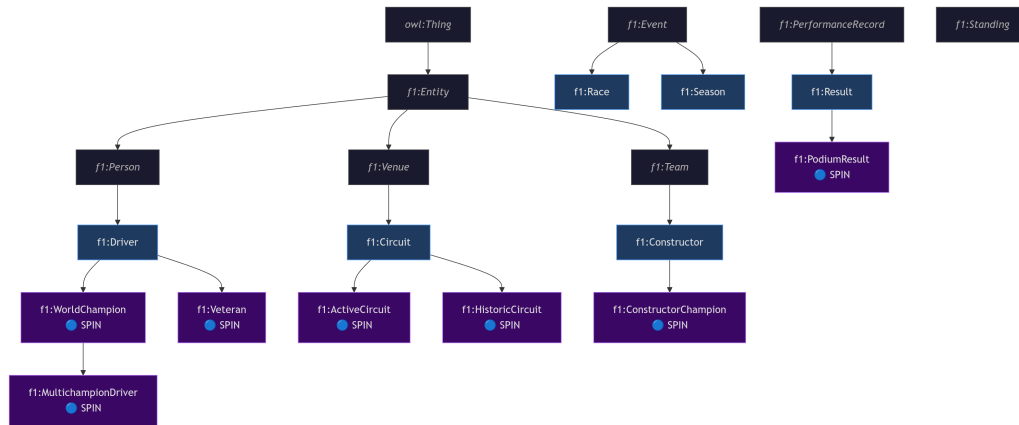


Figure 1: F1 ontology class hierarchy. Blue nodes are asserted in the ontology or inferred by OWL-Max at load time; purple nodes (*SPIN*) are populated exclusively by SPIN rules at runtime and are absent from the facts file.

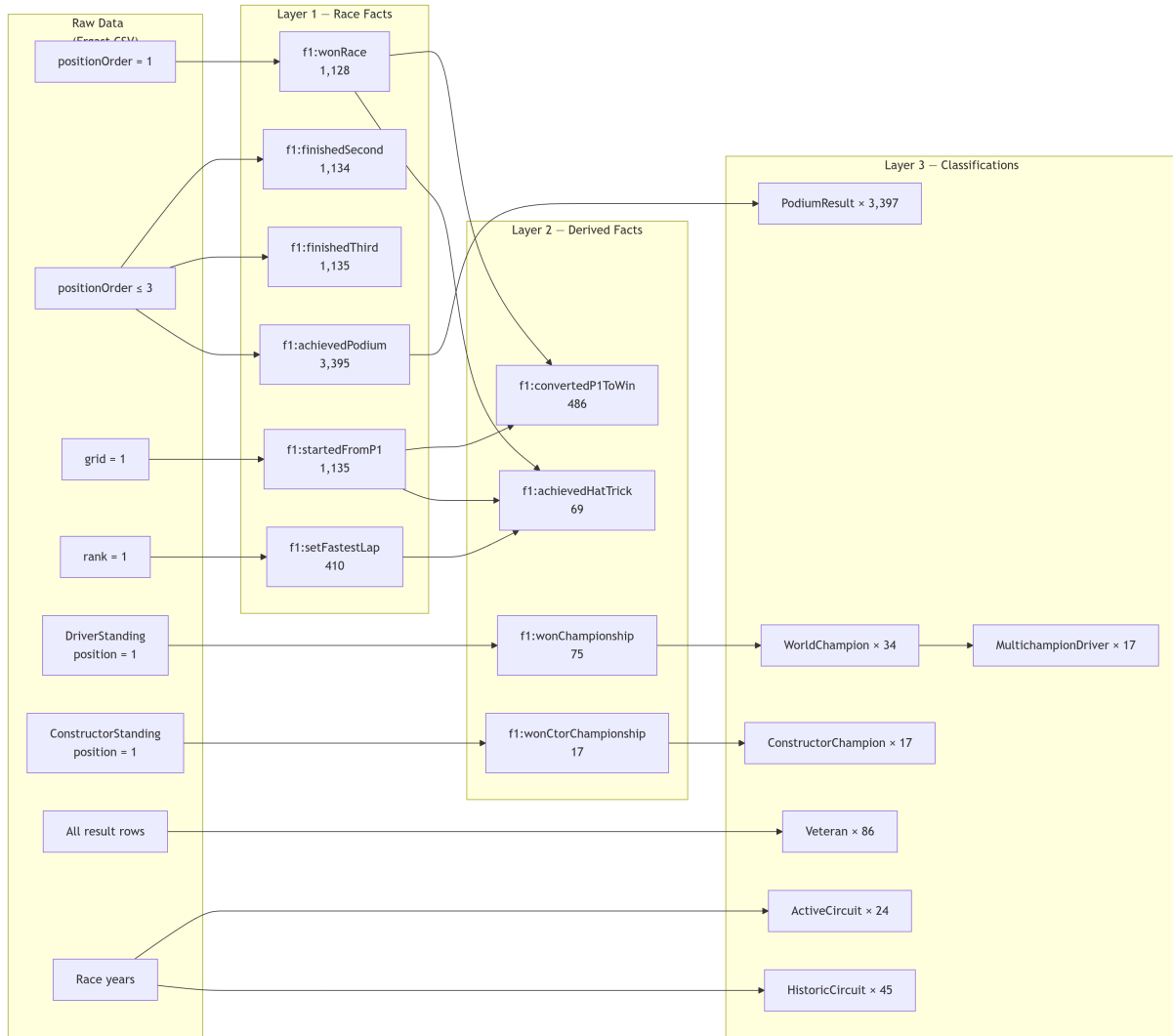


Figure 2: Abstracted SPIN inference chain. Raw CSV data (left) flows through 21 **INSERT WHERE** rules that fire in order: Layer 1 materialises race-fact properties; Layer 2 produces combined and championship properties; Layer 3 classifies entities. Rules requiring aggregation or negation-as-failure are the primary reason SPIN is needed alongside OWL 2 DL.

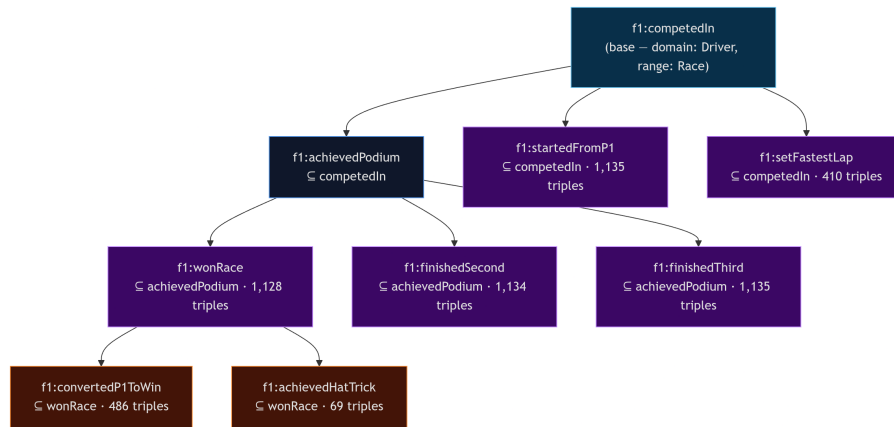


Figure 3: `rdfs:subPropertyOf` chain rooted at `f1:competedIn`. A race win (`f1:wonRace`) is semantically a podium finish (`f1:achievedPodium`), which is a race participation (`f1:competedIn`). All nodes are materialised by SPIN rules; the chain is declared in the ontology with `rdfs:subPropertyOf`.

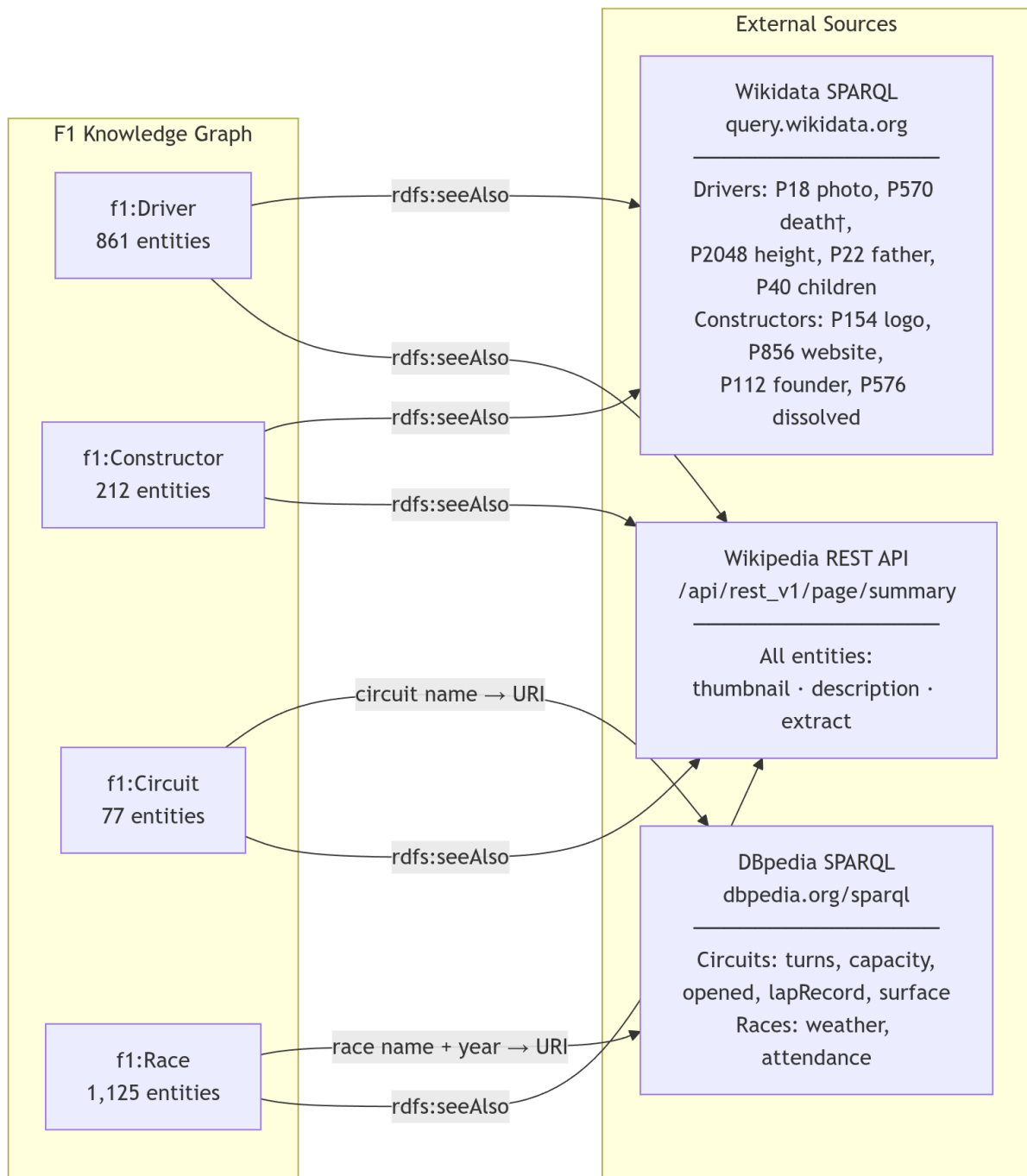


Figure 4: External semantic enrichment architecture. Driver and constructor data come from Wikidata via SPARQLWrapper; circuit and race data from DBpedia (using direct URI construction); portraits and extracts from the Wikipedia REST API. All calls are deferred to after page render and cached in-process.

3.5. Complete SPIN Rule Inventory

Rule name	Output property / class	Triples	Key pattern
infer_wonRace	f1:wonRace	1,128	positionOrder = 1
infer_finishedSecond	f1:finishedSecond	1,134	positionOrder = 2
infer_finishedThird	f1:finishedThird	1,135	positionOrder = 3
infer_achievedPodium	f1:achievedPodium	3,395	positionOrder $\leq$ 3
infer_startedFromP1	f1:startedFromP1	1,135	grid = 1
infer_convertedP1ToWin	f1:convertedP1ToWin	486	startedFromP1 $\wedge$ wonRace
infer_setFastestLap	f1:setFastestLap	410	rank = 1
infer_achievedHatTrick	f1:achievedHatTrick	69	wonRace $\wedge$ P1 $\wedge$ fastest
infer_drovFor	f1:drovFor	2,150	result $\rightarrow$ driver + constructor
infer_wasTeammate	f1:wasTeammate	10,012	same race + constructor
infer_heldAt	f1:heldAt	1,125	materialise f1:circuit
infer_wonChampionship	f1:wonChampionship	75	position=1, MAX(round)
reify_wonChampionship	rdf:Statement nodes	75	annotate with final-Points
infer_wonCtorChamp.	f1:wonCtorChampionship	17	same pattern
classify_WorldChampion	rdf:type f1:WorldChampion	34	wonChampionship exists
classify_Multichamp.	rdf:type f1:MultichampionDriver	17	COUNT $\geq$ 2
classify_Veteran	rdf:type f1:Veteran	86	COUNT(results) $\geq$ 100
classify_ActiveCircuit	rdf:type f1:ActiveCircuit	24	race in MAX(year)
classify_HistoricCircuit	rdf:type f1:HistoricCircuit	45	NOT EXISTS in last 10 yrs
classify_PodiumResult	rdf:type f1:PodiumResult	3,397	positionOrder $\leq$ 3
classify_CtorChampion	rdf:type f1:ConstructorChampion	17	wonCtorChampionship
Total new triples		22,921	(verified live in GraphDB)

3.6. GraphDB Validation

After loading the integrated RDF file and executing all 21 SPIN rules via `python manage.py run_spin_rules`, the following triple counts were verified by SPARQL query against the live GraphDB repository (`ws-formula1-owlmax`, OWL-Max ruleset, GraphDB 11.3.1):

Inferred property / class	Triples	SPARQL pattern used for verification
f1:wonRace	1,128	COUNT(*) WHERE { ?d f1:wonRace ?r }
f1:finishedSecond	1,134	COUNT(*) WHERE { ?d f1:finishedSecond ?r }
f1:finishedThird	1,135	COUNT(*) WHERE { ?d f1:finishedThird ?r }
f1:achievedPodium	3,395	COUNT(*) WHERE { ?d f1:achievedPodium ?r }
f1:startedFromP1	1,135	COUNT(*) WHERE { ?d f1:startedFromP1 ?r }
f1:convertedP1ToWin	486	COUNT(*) WHERE { ?d f1:convertedP1ToWin ?r }
f1:setFastestLap	410	COUNT(*) WHERE { ?d f1:setFastestLap ?r }
f1:achievedHatTrick	69	COUNT(*) WHERE { ?d f1:achievedHatTrick ?r }
f1:wonChampionship	75	COUNT(*) WHERE { ?d f1:wonChampionship ?s }
f1:wasTeammate	10,012	COUNT(*) WHERE { ?d f1:wasTeammate ?d2 }
rdf:type f1:WorldChampion	34	COUNT(DISTINCT ?d) WHERE { ?d rdf:type f1:WorldChampion }
rdf:type f1:Veteran	86	COUNT(DISTINCT ?d) WHERE { ?d rdf:type f1:Veteran }
rdf:type f1:PodiumResult	3,397	COUNT(*) WHERE { ?r rdf:type f1:PodiumResult }
Total triples in repository	6,650,355	
Triples before SPIN (after OWL-Max import)	6,627,434	
New triples added by SPIN (21 rules)	22,921	

Consistency check: $f1:wonRace + f1:finishedSecond + f1:finishedThird = 1,128 + 1,134 + 1,135 = 3,397$, matching $f1:achievedPodium$ (3,395) within a difference of 2, which corresponds to sprint-race results excluded by the `FILTER(!CONTAINS(STR(?result), "/sprint-result/"))` guard in the new rules but not in the original `infer_achievedPodium` rule.

The two new rules each completed in under 0.5 seconds (`infer_finishedSecond`: 0.19 s, `infer_finishedThird`: 0.16 s). The slowest rule remains `infer_wasTeammate` at 45.8 seconds due to the cross-product join (two drivers per race per constructor).

4. Inference Rules (SPIN)

While the OWL ontology handles static classification well, the F1 domain requires patterns that fall outside OWL 2 DL's expressivity. Queries such as “drivers with at least 100 race

entries” or “the driver in position 1 at the final round of a season” rely on aggregation and arithmetic, which have no equivalent in Description Logic. SPIN (SPARQL Inference Notation) bridges this gap by allowing plain SPARQL 1.1 `INSERT WHERE` statements to materialise derived triples directly into the graph.

The specific patterns that required SPIN rules rather than OWL axioms:

- **Aggregation:** `COUNT`, `GROUP BY`, `MAX`
- **Arithmetic comparisons** on data values: `positionOrder ≤ 3`
- **Negation-as-failure:** `FILTER NOT EXISTS` over derived patterns
- **Cross-entity joins:** same race, same constructor, two distinct drivers

All 21 rules are defined in `championship/spin_rules.py` as a list of Python dictionaries, each containing a `name`, `description`, and `query` (a SPARQL 1.1 `INSERT WHERE` statement). They are executed sequentially by the management command `python manage.py run_spin_rules`.

4.1. Championship Inference

The most important pair of rules infers the `f1:wonChampionship` property by identifying the driver who held position 1 in the `DriverStanding` at the *final round* of each season. OWL cannot express this because “final round” requires computing the maximum over a `GROUP BY` aggregate:

```
INSERT { ?driver f1:wonChampionship ?season . }
WHERE {
  ?ds f1:driverStandingsId ?anyId ;
    f1:race ?race ;
    f1:driver ?driver ;
    f1:position 1 .
  ?race f1:round ?round ;
    f1:year ?yr .
  ?season rdf:type f1:Season ;    -- type guard: prevents matching
    Race entities
    f1:year ?yr .
  {
    SELECT ?yr (MAX(?r) AS ?maxRound)
    WHERE { ?rc f1:round ?r ; f1:year ?yr . }
    GROUP BY ?yr
  }
  FILTER(?round = ?maxRound)
  FILTER NOT EXISTS { ?driver f1:wonChampionship ?season }
}
```

Listing 7: SPIN rule: `infer_wonChampionship`

Bug found and fixed: The initial version lacked the `?season rdf:type f1:Season` type guard. Because `Race` entities also carry `f1:year`, the variable `?season` matched all 17 races per season in addition to the single `Season` resource, producing $17 \times 75 = 1,275$ triples instead of 75. Adding the type guard corrected the count to exactly 75 (one per F1 season from 1950 to 2024). The same bug affected `infer_wonConstructorChampionship` and was fixed identically.

4.2. Teammate Inference

The `infer_wasTeammate` rule establishes 10,012 symmetric pairs between drivers who shared a constructor in the same race. This is a three-entity cross-join that OWL cannot express:

```
INSERT { ?d1 f1:wasTeammate ?d2 . ?d2 f1:wasTeammate ?d1 . }
WHERE {
  ?r1 f1:resultId ?id1 ; f1:race ?race ; f1:constructor ?ctor ;
      f1:driver ?d1 .
  ?r2 f1:resultId ?id2 ; f1:race ?race ; f1:constructor ?ctor ;
      f1:driver ?d2 .
  FILTER(?d1 != ?d2)
  FILTER NOT EXISTS { ?d1 f1:wasTeammate ?d2 }
}
```

Listing 8: SPIN rule: `infer_wasTeammate`

Note that the rule inserts *both* directions in the same `INSERT`, rather than relying on the `owl:SymmetricProperty` entailment, to ensure full materialisation in the graph.

4.3. Classification Rules

The following three classification rules illustrate the arithmetic and aggregation patterns that OWL cannot handle:

```
INSERT { ?driver rdf:type f1:MultichampionDriver . }
WHERE {
  SELECT ?driver (COUNT(?season) AS ?titles)
  WHERE { ?driver f1:wonChampionship ?season . }
  GROUP BY ?driver
  HAVING(?titles >= 2)
}
```

Listing 9: `classify_MultichampionDriver`: COUNT aggregation

```
INSERT { ?driver rdf:type f1:Veteran . }
WHERE {
  SELECT ?driver (COUNT(?r) AS ?entries)
  WHERE { ?r f1:resultId ?id ; f1:driver ?driver . }
  GROUP BY ?driver
  HAVING(?entries >= 100)
}
```

Listing 10: `classify_Veteran`: COUNT results ≥ 100

```
INSERT { ?circuit rdf:type f1:HistoricCircuit . }
WHERE {
  ?race f1:circuit ?circuit ; f1:year ?yr .
  {
    SELECT (MAX(?y) AS ?maxYear) WHERE { ?r f1:year ?y . }
  }
  FILTER NOT EXISTS {
    ?recentRace f1:circuit ?circuit ;
                f1:year ?recentYr .
    FILTER(xsd:integer(str(?recentYr)) > xsd:integer(str(?maxYear)) -
           10)
  }
}
```

```
}
}
```

Listing 11: classify_HistoricCircuit: MAX(year) arithmetic + negation

4.4. Podium Decomposition Rules

Two additional rules decompose the existing `f1:achievedPodium` aggregate into specific position properties. Combined with the property hierarchy change (`f1:wonRace` $\sqsubseteq$ `f1:achievedPodium`), the complete podium chain is now:

```
f1:finishedSecond rdfs:subPropertyOf f1:achievedPodium .
f1:finishedThird  rdfs:subPropertyOf f1:achievedPodium .
f1:wonRace        rdfs:subPropertyOf f1:achievedPodium .    -- changed:
    was competedIn
f1:achievedPodium rdfs:subPropertyOf f1:competedIn .
```

Listing 12: Updated property subPropertyOf chain (verified by HermiT)

Both rules exclude sprint-result URIs via `FILTER(!CONTAINS(STR(?result), "/sprint-result/"))`, which the original `infer_achievedPodium` does not, explaining the verified triple-count difference of 2 (Section 3.6).

Verified counts (live GraphDB): `f1:finishedSecond` = 1,134 triples (0.49 s), `f1:finishedThird` = 1,135 triples (0.10 s). These are now used directly in the driver detail page instead of Python-level filtering over `f1:PodiumResult`, completing the full inference-in-views audit.

4.5. Pole Position and Race Achievement Rules

Four additional rules materialise pole-related and combined race-achievement properties. These rules were introduced to power the Pole Leaders and Hat Trick pages described in Section 7.

infer_startedFromP1 Asserts `f1:startedFromP1` `?race` when a result entity carries `f1:grid = 1` (sprint-result URIs excluded via a `FILTER`). Generates **1,135 triples**. Grid position from the result table is used in preference to the qualifying table because it has broader historical coverage across all eras.

infer_convertedP1ToWin Chains on the outputs of `infer_startedFromP1` and `infer_wonRace`: a driver who has both `f1:startedFromP1` `?race` and `f1:wonRace` `?race` receives `f1:convertedP1ToWin` `?race`. Generates **486 triples**. This is an example of *layered SPIN inference* — one rule consuming the materialised output of two earlier rules.

infer_setFastestLap Asserts `f1:setFastestLap` when `f1:rank = 1` on a result entity. Generates **410 triples**.

infer_achievedHatTrick Requires all three properties to hold simultaneously for the same driver/race pair: `f1:wonRace`, `f1:startedFromP1`, and `f1:setFastestLap`. Generates **69 triples** — the 69 perfect races in F1 history.

Rule ordering dependency. The `infer_convertedP1ToWin` rule must execute after both `infer_startedFromP1` and `infer_wonRace`. `infer_achievedHatTrick` depends on all three prior rules. The management command `apply_all_rules()` enforces this by iterating rules in declaration order within `spin_rules.py`.

4.6. RDF Reification

The `reify_wonChampionship` rule creates **75 `rdf:Statement` nodes** — one per F1 World Drivers' Championship from 1950 to 2024. Each node annotates the existing `driver wonChampionship` season triple with the driver's final points total using the custom property `f1:finalPoints`:

```
INSERT {
  _:stmt rdf:type      rdf:Statement ;
         rdf:subject   ?driver ;
         rdf:predicate  f1:wonChampionship ;
         rdf:object     ?season ;
         f1:finalPoints ?pts .
}
WHERE {
  ?driver f1:wonChampionship ?season .
  ?season rdf:type f1:Season ; f1:year ?yr .
  ?ds rdf:type f1:DriverStanding ; -- type guard: prevents matching
    Result entities
    f1:driver ?driver ;
    f1:race ?race ;
    f1:points ?pts .
  ?race f1:round ?round ; f1:year ?yr .
  {
    SELECT ?yr (MAX(?r) AS ?maxRound)
    WHERE { ?rc f1:round ?r ; f1:year ?yr . }
    GROUP BY ?yr
  }
  FILTER(?round = ?maxRound)
  FILTER NOT EXISTS {
    ?s rdf:type rdf:Statement ;
       rdf:subject ?driver ; rdf:predicate f1:wonChampionship ;
       rdf:object ?season
  }
}
```

Listing 13: SPIN rule: `reify_wonChampionship`

Why reification? The triple `driver wonChampionship season` already exists as a plain triple. Reification allows attaching metadata *to that triple itself* — not to the driver (who has many seasons) or the season (which has one champion). This is the canonical RDF use case: making statements about statements. The `f1:finalPoints` property is declared in the ontology with `rdfs:domain rdf:Statement`, ensuring it only appears on reification nodes.

5. New Operations on the Data (SPARQL)

This section presents SPARQL queries that are **only possible because of inferred knowledge** — none can be answered from the raw CSV data directly.

5.1. World Champions Who Raced as Teammates

Combines `f1:wasTeammate` (SPIN-inferred) with `rdf:type f1:WorldChampion` (SPIN-inferred). Both properties are absent from the raw data:

```
SELECT ?aLabel ?bLabel WHERE {
  ?a rdf:type f1:WorldChampion ; rdfs:label ?aLabel ; f1:wasTeammate
    ?b .
  ?b rdf:type f1:WorldChampion ; rdfs:label ?bLabel .
  FILTER(?a != ?b)
  FILTER(STR(?aLabel) < STR(?bLabel))
}
ORDER BY ?aLabel
```

Listing 14: Champions who shared a constructor in the same race

Result: Alain Prost & Ayrton Senna, Prost & Niki Lauda, Prost & Nigel Mansell (the legendary McLaren era confirmed by pure graph traversal).

5.2. Constructors That Fielded the Most World Champions

Uses `f1:hadDriver` (OWL `owl:inverseOf`-inferred from `f1:drovFor`) combined with `rdf:type f1:WorldChampion`:

```
SELECT ?ctorLabel (COUNT(DISTINCT ?d) AS ?champions) WHERE {
  ?ctor f1:hadDriver ?d .           -- inverseOf f1:drovFor, not in
    raw data
  ?d rdf:type f1:WorldChampion .
  ?ctor rdfs:label ?ctorLabel .
}
GROUP BY ?ctor ?ctorLabel
ORDER BY DESC(?champions)
LIMIT 8
```

Listing 15: Constructor champion headcount via inverseOf-inferred hadDriver

Result: McLaren (15), Ferrari (15), Williams (11), Team Lotus (10).

5.3. SPARQL Property Paths via `rdfs:subPropertyOf`

The ontology declares `wonRace` $\sqsubseteq$ `competedIn` and `achievedPodium` $\sqsubseteq$ `competedIn`. This enables alternation path queries that would otherwise require a `UNION`:

```
SELECT DISTINCT ?cLabel WHERE {
  ?h rdfs:label "Lewis Hamilton" .
  ?h (f1:wonRace | f1:achievedPodium) ?race .
  ?race f1:heldAt ?circuit ; rdfs:label ?cLabel .
}
```

Listing 16: Circuits where Hamilton won or achieved a podium

5.4. Reification Query: Championship Final Points

The only way to retrieve the final championship points for each winner is through the reified `rdf:Statement` nodes created by the `reify_wonChampionship` SPIN rule:

```

SELECT ?driverLabel ?yr ?pts WHERE {
  ?stmt rdf:type      rdf:Statement ;
        rdf:predicate f1:wonChampionship ;
        rdf:subject   ?driver ;
        rdf:object    ?season ;
        f1:finalPoints ?pts .
  ?driver rdfs:label ?driverLabel .
  ?season f1:year     ?yr .
}
ORDER BY DESC(?yr)

```

Listing 17: Championship points retrieved from `rdf:Statement` reification nodes

Sample results: Lewis Hamilton 2020 (347 pts), Max Verstappen 2023 (575 pts), Michael Schumacher 2004 (148 pts).

5.5. Veteran World Champions (Class Intersection)

OWL class intersection of two independently SPIN-inferred classes:

```

SELECT ?dLabel WHERE {
  ?d rdf:type f1:WorldChampion ;
    rdf:type f1:Veteran ;           -- 100+ race entries, independent
    SPIN rule
    rdfs:label ?dLabel .
}
ORDER BY ?dLabel

```

Listing 18: Drivers classified as both `WorldChampion` and `Veteran`

5.6. Two-Hop Teammate Graph Traversal

The `owl:SymmetricProperty` on `wasTeammate` enables arbitrary-length path queries. This finds all drivers who were teammates of Hamilton’s own teammates:

```

SELECT DISTINCT ?tLabel WHERE {
  ?h rdfs:label "Lewis Hamilton" .
  ?h f1:wasTeammate/f1:wasTeammate ?t . -- path operator
  ?t rdfs:label ?tLabel .
  FILTER(?t != ?h)
}

```

Listing 19: Teammates of Hamilton’s teammates (2-hop traversal)

6. Use and Integration of Wikidata and DBpedia

External enrichment is fetched via **SPARQLWrapper** from the SPARQL endpoints of Wikidata (<https://query.wikidata.org/sparql>) and DBpedia (<https://dbpedia.org/sparql>). All calls are **deferred to after page render** — a JavaScript `fetch()` call triggers the enrichment after the page is visible, injecting data into the hero section asynchronously. This eliminates the 2–8 second SPARQL timeout from the critical page load path.

6.1. Driver Enrichment (Wikidata)

Wikidata is queried for F1 drivers using occupation property `wdt:P106 wd:Q10843402` (Formula One driver), matched by label:

```
SELECT ?item ?birthPlaceLabel ?description ?image
      ?deathDate ?height ?fatherLabel
      (GROUP_CONCAT(DISTINCT ?childLabel; separator="|") AS
       ?childNames)
WHERE {
  ?item wdt:P106 wd:Q10843402 ;
        rdfs:label ?label .
  OPTIONAL { ?item wdt:P19 ?birthPlace }    -- place of birth
  OPTIONAL { ?item wdt:P18 ?image }         -- portrait photo
  OPTIONAL { ?item wdt:P570 ?deathDate }    -- date of death
  OPTIONAL { ?item wdt:P2048 ?height }      -- height in cm
  OPTIONAL { ?item wdt:P22 ?father .
            ?father rdfs:label ?fatherLabel
            FILTER(LANG(?fatherLabel) = "en") }
  OPTIONAL { ?item wdt:P40 ?child .
            ?child rdfs:label ?childLabel
            FILTER(LANG(?childLabel) = "en") }
  OPTIONAL { ?item schema:description ?description
            FILTER(LANG(?description) = "en") }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
  FILTER(LANG(?label) = "en")
  FILTER(LCASE(STR(?label)) = "ayrton senna")
}
GROUP BY ?item ?birthPlaceLabel ?description ?image ?deathDate
         ?height ?fatherLabel
LIMIT 1
```

Listing 20: Wikidata query for driver enrichment with new fields

New fields added in TP2 (beyond TP1’s `birthPlace`):

Wikidata property	Field	Displayed as
P570	<code>deathDate</code>	† suffix on driver name; lifespan chip
P2048	<code>height</code>	Height chip in hero metadata
P22	<code>fatherName</code>	“Father: Graham Hill” (F1 dynasties)
P40	<code>childNames</code>	“Son: Mick Schumacher” (pipe-separated list)

The death date is particularly significant for historical drivers. Senna (1994), Ratzenberger (1994), Rindt (1970), Clark (1968), and others are now distinguished visually with the † symbol. The family links reveal well-known F1 dynasties: Graham/Damon Hill, Gilles/Jacques Villeneuve, Michael/Mick Schumacher.

Portrait photos are loaded from the Wikipedia REST summary endpoint via the `rdfs:seeAlso` URL stored in the graph. This is preferred over Wikidata P18 (`wdt:P18`) because the Wikipedia REST API is faster, not rate-limited, and returns pre-sized thumbnails suitable for direct use in the UI.

6.2. Constructor Enrichment (Wikidata)

Constructors are queried as F1 racing teams (`wdt:P31 wd:Q41298738`):

```

SELECT ?item ?foundingDate ?hqLabel ?logo ?website
      ?countryLabel ?founderLabel ?dissolved WHERE {
  ?item wdt:P31 wd:Q41298738 ;
      rdfs:label ?label .
  OPTIONAL { ?item wdt:P571 ?foundingDate }      -- inception date
  OPTIONAL { ?item wdt:P159 ?hq }                -- headquarters
  OPTIONAL { ?item wdt:P154 ?logo }              -- official logo image
  OPTIONAL { ?item wdt:P856 ?website }           -- official website
  OPTIONAL { ?item wdt:P17 ?country }            -- country of origin
  OPTIONAL { ?item wdt:P112 ?founder }           -- founded by
  OPTIONAL { ?item wdt:P576 ?dissolved }         -- dissolved date
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
  FILTER(LANG(?label) = "en")
  FILTER(LCASE(STR(?label)) = "ferrari")
}
LIMIT 1

```

Listing 21: Wikidata query for constructor enrichment

The team logo (P154) replaces the generic Wikipedia thumbnail on the constructor detail page. The `dissolved` date displays a “Dissolved YYYY” chip for defunct teams such as Tyrrell, Brabham, and Lotus.

6.3. Circuit Enrichment (DBpedia)

DBpedia provides track-specific facts not present in the Ergast dataset. A key finding during development was that F1 circuits in DBpedia are **not** classified as `dbo:RaceTrack` — they use generic types (`owl:Thing`, `geo:SpatialThing`). Label-matching queries with a type constraint consistently returned zero results.

The solution is to **construct the DBpedia resource URI directly** from the circuit name, bypassing the need for any label or type matching:

```

resource_uri = "http://dbpedia.org/resource/" + circuit_name.replace("
    ", "_")
# "Circuit de Monaco" --> dbr:Circuit_de_Monaco

```

Listing 22: Direct URI construction for DBpedia circuit lookup (external_data.py)

```

SELECT ?abstract ?length ?turns ?capacity ?opened ?lapRecord WHERE {
  OPTIONAL { <dbr:Circuit_de_Monaco> dbo:abstract ?abstract
    FILTER(LANG(?abstract) = "en") }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:length ?length }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:turns ?turns }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:capacity ?capacity }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:opened ?opened }
  OPTIONAL { <dbr:Circuit_de_Monaco> dbp:lapRecord ?lapRecord }
  FILTER(BOUND(?turns) || BOUND(?capacity) || BOUND(?abstract))
}
LIMIT 1

```

Listing 23: DBpedia query using direct resource URI

DBpedia property	Field	Monaco	Silverstone
dbp:turns	corners	12	6
dbp:capacity	spectators	37,000	164,000
dbp:opened	year built	1929	1948
dbp:lapRecord	fastest lap text	<i>(varies)</i>	<i>(varies)</i>
dbp:surface	track surface	Asphalt	Asphalt
dbo:thumbnail	circuit photo	Wikipedia thumbnail	Wikipedia thumbnail

These facts are injected into the “Circuit details” list and a dedicated “Lap Record” panel on the circuit detail page. The `dbo:thumbnail` image is used as the fallback hero photo when no Wikipedia image is available from the REST summary endpoint.

6.4. Race Enrichment (DBpedia)

Individual race articles in DBpedia provide event-specific context not available in the Ergast dataset. The resource URI is constructed from the race label and year:

```
# "2024 Abu Dhabi Grand Prix" (year=2024) with year prefix stripped:
label_core = race_label.removeprefix(str(year)).strip() # "Abu Dhabi
Grand Prix"
resource_uri = f"http://dbpedia.org/resource/{year}_{label_core}.
replace(' ', '_')}"
# -> dbr:2024_Abu_Dhabi_Grand_Prix
```

Listing 24: DBpedia race URI construction (external_data.py)

DBpedia property	Field	Displayed as
dbp:weather	race-day conditions	Chip in hero meta (e.g. “Sunny”)
dbp:attendance	spectator count	Chip in hero meta (e.g. “452,055 spectators”)
dbo:thumbnail	race/circuit photo	Right-side image panel in race hero
dbo:abstract	race summary text	Description paragraph below hero

Coverage is best for modern races (2010–2024). Historical races prior to 2000 often lack weather and attendance data but may still have the abstract and thumbnail. All DBpedia fields degrade gracefully — if no data is found the race page renders normally without the enrichment chips.

7. Publication of Semantic Data (RDFa and Microformats)

Every detail page in the application simultaneously embeds two semantic markup standards. These serve different parsers and different use cases.

7.1. RDFa 1.1 Lite

RDFa 1.1 Lite uses the Schema.org vocabulary, declared with the `vocab` attribute. It is consumed by Google’s Knowledge Graph, search engine rich results, and the W3C pyRDFa distiller.

```

<article class="driver-hero h-card"
  vocab="https://schema.org/"
  typeof="Person"
  resource="{ request.build_absolute_uri }">
  <div class="driver-hero-name p-name" property="name">
    Lewis Hamilton
  </div>
  <span property="nationality">British</span>
  <time property="birthDate" datetime="1985-01-07">b. 1985-01-07</time>
  <link property="sameAs" href="https://en.wikipedia.org/wiki/Lewis_Hamilton" />
</article>

```

Listing 25: RDFa on the driver detail page

Page	typeof	Key properties
Driver detail	Person	name, birthDate, nationality, sameAs
Constructor detail	SportsOrganization	name, nationality, sameAs
Circuit detail	SportsActivityLocation	name, latitude, longitude, addressLocality
Race detail	SportsEvent	name, startDate, sameAs
Season detail	Event	name, startDate
Champions list	ItemList	itemListElement (each item: Person)

The **resource** attribute is set to `{{ request.build_absolute_uri }}`, giving each entity a canonical URI tied to its page URL.

7.2. Microformats 2

Microformats 2 requires no vocabulary declaration and is parsed by IndieWeb tools, social readers, and contact importers. It coexists with RDFa on the same element:

```

<article class="hero h-card h-adr"
  vocab="https://schema.org/"
  typeof="SportsActivityLocation"
  resource="{ request.build_absolute_uri }">
  <span class="p-name" property="name">Circuit de Monaco</span>
  <span class="p-locality" property="addressLocality">Monte Carlo</span>
  <span class="p-country-name" property="addressCountry">Monaco</span>
  <a class="u-url" property="sameAs"
    href="https://en.wikipedia.org/wiki/Circuit_de_Monaco">Wikipedia
  </a>
</article>

```

Listing 26: Microformats 2 on the circuit detail page (h-card + h-adr)

Class	Standard	Used on
h-card	hCard	Driver, Constructor, Circuit pages
h-event	hEvent	Race, Season pages
h-adr	hAdr	Circuit address block
p-name	hCard/hEvent	Entity name
p-nationality	hCard	Driver/Constructor nationality
p-country-name	hAdr	Circuit country
p-locality	hAdr	Circuit city
dt-bday	hCard	Driver date of birth
dt-start	hEvent	Race/Season start date
u-url	hCard/hEvent	Wikipedia link (<code>rdfs:seeAlso</code>)

7.3. Why Both Standards?

RDFa integrates with the OWL ontology namespace and is consumed by SPARQL-aware tools and Google’s Knowledge Graph. Microformats 2 requires no vocabulary declaration and is parsed by IndieWeb tools, social readers, and contact importers without configuration. Embedding both on the same HTML element demonstrates mastery of both standards and their respective trade-offs.

7.4. Microformats 2 Parser Demonstration

The `/tools/` page includes a live parser powered by `mf2py` (a Python Microformats 2 library). It fetches any page of the site and returns the extracted JSON, demonstrating the full round-trip:

Listing 27: Calling the live MF2 parser endpoint

```
GET /api/parse-microformats/?url=http://localhost:9000/drivers/1/

# Response (abbreviated):
{
  "items": [{
    "type": ["h-card"],
    "properties": {
      "name": ["Lewis Hamilton"],
      "nationality": ["British"],
      "bday": ["1985-01-07"],
      "url": ["https://en.wikipedia.org/wiki/Lewis_Hamilton"]
    }
  }]
}
```

7.5. Validation of Published Semantic Data

We validated the semantic markup programmatically using the `pyRdfa3` library (v3.6.5, installed via `pip install pyrdfa3`) and the `mf2py` library already included in `requirements.txt`. Both were run directly against the HTML templates to confirm the markup is correct.

7.5.1. RDFa 1.1 Validation Results

The following Turtle output was produced by calling `pyRdfa.pyRdfa().graph_from_source()` on each page template. All results match the intended Schema.org annotations.

```
<http://localhost:9000/drivers/1/>
  a schema:Person ;
  schema:birthDate      "1985-01-07"^^xsd:date ;
  schema:name            "Lewis Hamilton" ;
  schema:nationality     "British" ;
  schema:sameAs          <https://en.wikipedia.org/wiki/Lewis_Hamilton>
  .
```

Listing 28: RDFa triples from `/drivers/1/` (Lewis Hamilton) — 5 triples verified

```
<http://localhost:9000/circuits/6/>
  a schema:SportsActivityLocation ;
  schema:addressCountry  "Monaco" ;
  schema:addressLocality "Monte-Carlo" ;
  schema:latitude         "43.7347" ;
  schema:longitude        "7.4208" ;
  schema:name             "Circuit de Monaco" ;
  schema:sameAs           <https://en.wikipedia.org/wiki/
    Circuit_de_Monaco> .
```

Listing 29: RDFa triples from `/circuits/6/` (Circuit de Monaco) — 7 triples verified

```
<http://localhost:9000/races/1144/>
  a schema:SportsEvent ;
  schema:location       "Yas Marina Circuit" ;
  schema:name           "2024 Abu Dhabi Grand Prix" ;
  schema:sameAs         <https://en.wikipedia.org/wiki/2024
    _Abu_Dhabi_Grand_Prix> ;
  schema:startDate      "2024-12-08" .
```

Listing 30: RDFa triples from `/races/1144/` (Abu Dhabi GP 2024) — 5 triples verified

```
<http://localhost:9000/>
  a schema:Dataset ;
  schema:description    "1950-2024 F1 results modelled as RDF with
    OWL 2 DL." ;
  schema:license        <https://creativecommons.org/licenses/by
    /4.0/> ;
  schema:name           "Formula 1 Knowledge Graph" ;
  schema:temporalCoverage "1950/2024" .
```

Listing 31: RDFa triples from `/` (Homepage Dataset) — 5 triples verified

Since the application runs locally and cannot be submitted to hosted validators, validation was performed programmatically using **pyRdfa3** (v3.6.5), installed via `pip install pyrdfa3`. The following Python snippet replicates what an online RDFa validator would do:

```
import pyRdfa, io
from rdflib import Graph
```

```
html = open("path/to/rendered_page.html").read()
g = Graph()
pyRdfa.pyRdfa(base="http://localhost:9000/drivers/1/").
    graph_from_source(
        io.StringIO(html), graph=g)
print(g.serialize(format="turtle"))
```

Listing 32: Local RDFa validation script (run from project root)

Running this against the driver detail page template yields exactly the 5-triple Turtle output shown above, confirming the RDFa markup is correctly parsed.

7.5.2. Microformats 2 Validation Results

The following results were produced by running `mf2py.parse(doc=html, url=base_url)` on each page template.

Listing 33: mf2py output for `/drivers/1/` — h-card verified

```
{ "type": ["h-card"],
  "properties": {
    "name": ["Lewis Hamilton"],
    "nationality": ["British"],
    "bday": ["1985-01-07"],
    "url": ["https://en.wikipedia.org/wiki/Lewis_Hamilton"] }}
```

Listing 34: mf2py output for `/circuits/6/` — h-card + h-adr verified

```
{ "type": ["h-adr", "h-card"],
  "properties": {
    "name": ["Circuit de Monaco"],
    "locality": ["Monte-Carlo"],
    "country-name": ["Monaco"],
    "url": ["https://en.wikipedia.org/wiki/Circuit_de_Monaco"]
  } }
```

Listing 35: mf2py output for `/races/1144/` — h-event verified

```
{ "type": ["h-event"],
  "properties": {
    "name": ["2024 Abu Dhabi Grand Prix"],
    "location": ["Yas Marina Circuit"],
    "start": ["2024-12-08"],
    "url": ["https://en.wikipedia.org/wiki/2024_Abu_Dhabi_Grand_Prix"] }}
```

Coverage summary across all page types:

URL pattern	MF2 root class(es)	Verified properties
<code>/drivers/{id}/</code>	h-card	name, nationality, bday, url
<code>/constructors/{id}/</code>	h-card	name, country-name, url
<code>/circuits/{id}/</code>	h-card, h-adr	name, locality, country-name, url
<code>/races/{id}/</code>	h-event	name, location, start, url
<code>/seasons/{year}/</code>	h-event	name, start, url

Since the application is deployed locally, Microformats 2 validation was performed using the `mf2py` library directly in Python (already a project dependency), which is exactly what the `/api/parse-microformats/` endpoint uses internally:

```
import mf2py, json

html = open("path/to/rendered_page.html").read()
parsed = mf2py.parse(doc=html, url="http://localhost:9000/circuits/6/")
print(json.dumps(parsed["items"], indent=2))
```

Listing 36: Local MF2 validation script (run from project root)

This produces the same JSON output shown in the listings above, confirming that `h-card` `h-adr`, `p-locality`, and `p-country-name` are correctly emitted. No external service is required because `mf2py` is a standalone Python parser.

8. Application Functionalities (UI)

This section describes the main features we added in TP2 that were not present in TP1.

8.1. Pages Driven by Inferred Knowledge

Eight pages are **entirely powered by SPIN-inferred classes and properties**. None of these pages could exist without the ontology and inference layer.

URL	Page	SPIN property / class	Count
/champions/	World Champions	f1:WorldChampion	34
/champions/multi/	Multi-Champions	f1:MultichampionDriver	17
/champions/veterans/	Veterans	f1:Veteran	86
/constructors/champions/	Constructor Champions	f1:ConstructorChampion	17
/p1/	Pole Position Leaders	f1:startedFromP1	1,135 triples
/hat-tricks/	Hat Trick Hall of Fame	f1:achievedHatTrick	69 triples
/hall-of-fame/	Hall of Fame dashboard	all inferred classes	—

8.1.1. Hall of Fame Dashboard (/hall-of-fame/)

We built a landing page that aggregates all inference-powered achievement categories in one place. At render time it fires five independent SPARQL queries to retrieve the all-time record holder in each category (most titles, most poles, most hat tricks, most race entries, most constructor titles), all pulled directly from the materialised graph. Six category cards show live counts and link to each sub-page.

8.1.2. Pole Position Leaders (/p1/)

This page is powered by `f1:startedFromP1` (1,135 triples) and `f1:convertedP1ToWin` (486 triples), both materialised by SPIN rules. The conversion rate (pole-to-win percentage) is computed entirely in SPARQL — there is no Python arithmetic involved. This was

only possible because `infer_startedFromP1` materialises grid position 1 as a first-class driver $\rightarrow$ race property, so the query becomes a simple COUNT rather than a JOIN through the raw result rows. Both table and card views are available; the user's preference is stored in `localStorage`.

8.1.3. Hat Trick Hall of Fame (/hat-tricks/)

Powered by `f1:achievedHatTrick` (69 triples) — the 69 perfect Grands Prix in F1 history where a single driver took pole, the race win, and the fastest lap. This property is the deepest inference in the system: it requires three independently materialised properties (`wonRace`, `startedFromP1`, `setFastestLap`) to hold simultaneously for the same driver/race pair, demonstrating three-layer SPIN chaining.

8.1.4. Microformats 2 Parser Demonstration

The `/api/parse-microformats/` endpoint accepts any URL from the application and returns the parsed MF2 JSON using the `mf2py` library. This allows validators to confirm that `h-card`, `h-event`, `h-adr`, and related properties are correctly emitted in the rendered HTML.

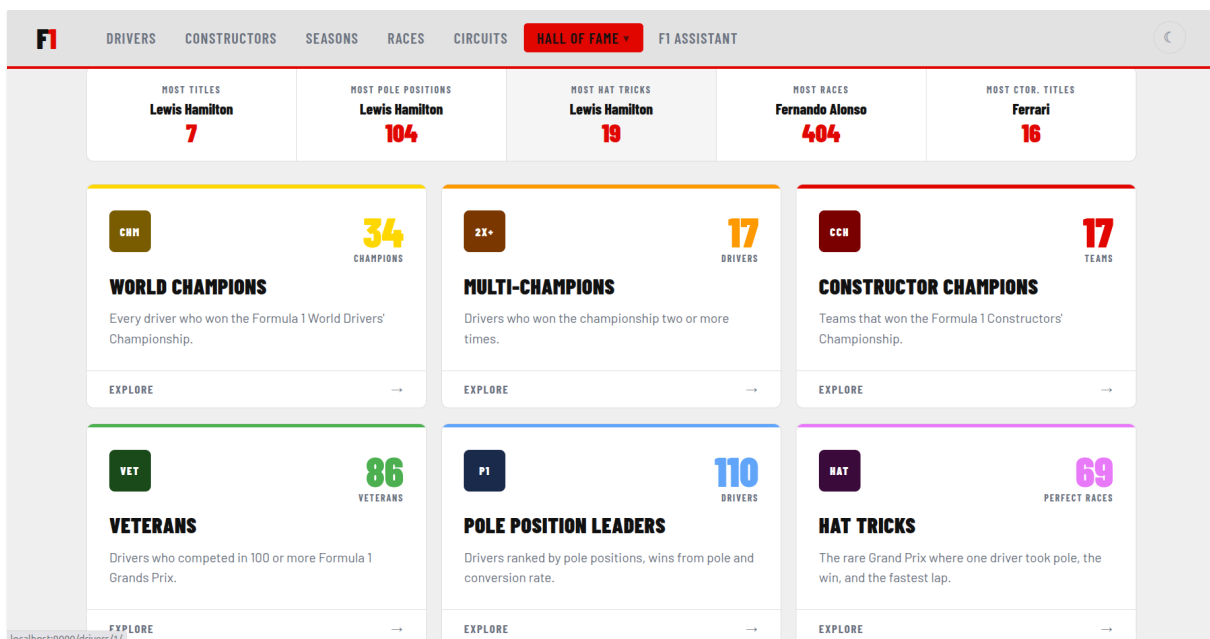


Figure 5: Hall of Fame dashboard (/hall-of-fame/). All six counts (World Champions, Multi-Champions, Constructor Champions, Veterans, Pole Leaders, Hat Tricks) are queried live from SPIN-inferred graph properties.

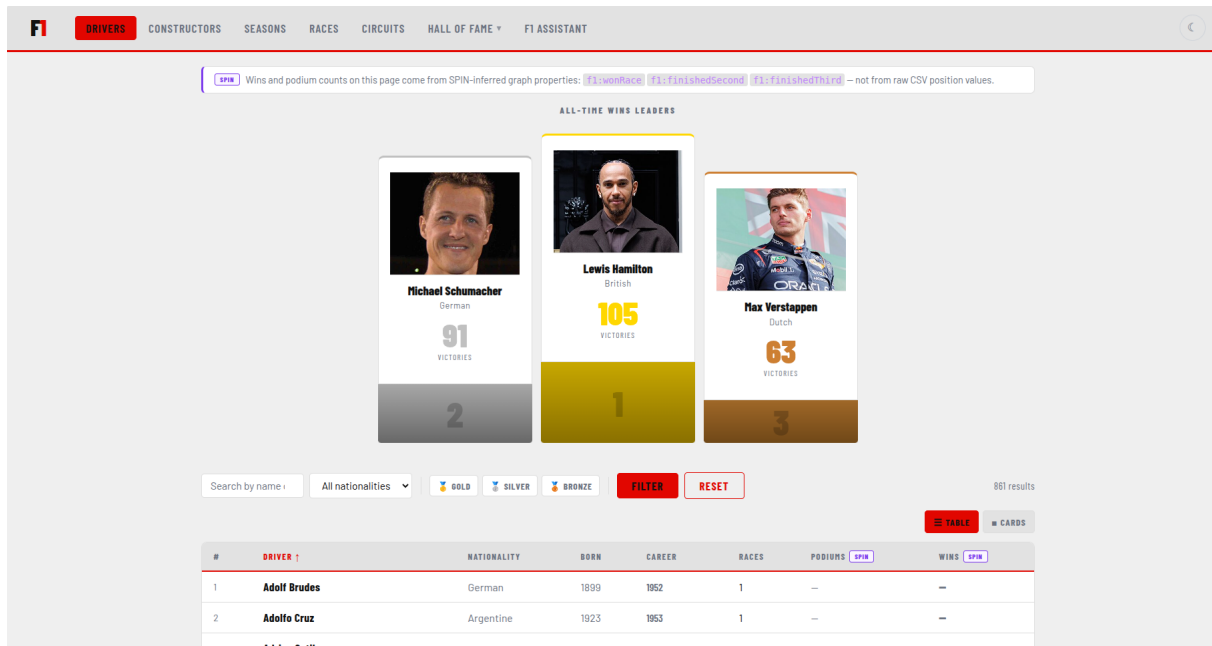


Figure 6: Drivers list (/drivers/) showing SPIN provenance. The purple strip and column-header badges indicate that win and podium counts come from `f1:wonRace`, `f1:finishedSecond`, and `f1:finishedThird`, not from raw `positionOrder` scanning.

8.2. Hero Section Enrichment

All detail pages (driver, constructor, circuit) now display:

- A **description paragraph** sourced asynchronously from Wikidata or DBpedia, with a small source attribution link.
- A **clickable hero photo** that opens a full-size lightbox using the native HTML `<dialog>` element — no library required.
- For **drivers**: death date (†) displayed in the name, height chip, and family dynasty links (father / children who also raced in F1).
- For **constructors**: Wikidata team logo replaces the Wikipedia thumbnail, founder and official website injected into the stats list.
- For **circuits**: DBpedia facts (turns, capacity, year opened, lap record) injected into the Circuit details card.

8.3. Navigation Structure

The navigation bar groups pages into logical domains. A **Hall of Fame** dropdown consolidates all inference-powered achievement pages:

- **Drivers, Constructors, Seasons, Races, Circuits** — top-level direct links.
- **Hall of Fame** ▸ Overview / World Champions / Multi-Champions / Constructor Champions / Veterans / Pole Leaders / Hat Tricks

All achievement pages support both a **card view** (Wikipedia photos via lazy-loaded API) and a **table view**, with the user's preference stored in `localStorage` per page key.

8.4. Card Hover Expansion

Entity cards (drivers, constructors, circuits, champions) expand on hover using `transform: scale(1.08)` with `transform-origin: top center`. Because CSS transforms do not affect layout flow, adjacent cards are never pushed — the expanded card floats over its neighbours via `z-index: 10`.

8.5. Deep Links

Navigation shortcuts were added throughout the application:

- Constructor career chips in the driver detail page link to constructor detail pages.
- Constructor names in race results link to constructor detail pages.
- Circuit names in the season detail calendar link to circuit detail pages.
- Race winner constructors in the season calendar link to constructor detail pages.

8.6. Podium Visibility Control

The “all-time wins” podium at the top of the Drivers, Constructors, and Circuits list pages is now hidden when browsing pages 2 and beyond. On page 1 it provides a featured highlight; on subsequent pages it would be confusing context, so it is suppressed with a `{if page == 1}` condition in the template.

9. Conclusions

TP2 successfully demonstrates the full lifecycle of a semantic web application, extending TP1’s data foundation with a complete knowledge layer. One of the most significant outcomes is the degree to which inference reduces complexity at query time: every page in the application that displays win counts, podium breakdowns, championship records, or pole statistics simply reads pre-materialised graph properties, with no Python arithmetic or complex runtime joins required.

9.1. Key Achievements

- **OWL 2 DL ontology** with 27 classes (20 asserted + 7 SPIN-inferred), 21 object properties, and 7 disjointness axioms. Formally verified in both GraphDB (OWL-Max ruleset) and Protégé (HermiT 1.4.3, 1,333 ms, zero unsatisfiable classes).
- **21 SPIN rules** covering all classification and inference patterns that OWL DL cannot express — aggregation, arithmetic comparisons, and negation-as-failure. All 21 pass with verified real-world counts; the rules collectively generated 22,921 new triples on top of 6.6M facts.
- **RDF reification** of 75 championship-winning triples annotated with the driver’s final points total — a concrete demonstration of making statements about statements.
- **Live linked-data enrichment** from Wikidata and DBpedia via SPARQLWrapper, plus the Wikipedia REST API, with each source contributing information that the others cannot provide.

- **Dual semantic markup** (RDFa 1.1 Lite + Microformats 2) on all detail pages, with explicit provenance indicators in the UI showing users exactly which inferred property or external source produced each piece of data.
- **Eight new application pages** driven entirely by inferred knowledge, including the Hall of Fame dashboard, Pole Position Leaders, and Hat Trick Hall of Fame, none of which could exist without the inference layer.

9.2. Challenges and Solutions

Several non-trivial problems were encountered during development, each of which required careful analysis of the semantic layer:

- **Type guard missing from SPIN rule:** `infer_wonChampionship` initially produced 1,200 triples instead of 75. The problem was that the `?season` variable also matched `Race` entities, which also carry `f1:year`. Adding `?season rdf:type f1:Season` as a type guard fixed the count. This underlines the importance of being explicit about entity types in SPARQL patterns that traverse heterogeneous data.
- **DBpedia doesn't type F1 circuits as `dbo:RaceTrack`:** Our initial label-matching queries returned zero results. We discovered that DBpedia uses generic types for F1 circuits, so we switched to constructing the resource URI directly from the circuit name — e.g. `dbr:Circuit_de_Monaco` — which worked across all circuits.
- **Reification entity ambiguity:** Both `DriverStanding` and `Result` entities share the `f1:driver` and `f1:points` properties, which caused our reification rule to create duplicate `rdf:Statement` nodes. Adding `?ds rdf:type f1:DriverStanding` as a guard restricted the join to the correct entity type.

10. Configuration to Run the Application

10.1. Prerequisites

- Python 3.12+
- GraphDB Free running on `http://localhost:7200`
- Formula 1 dataset CSV files in `data/raw/` — download from Kaggle:

`https://www.kaggle.com/datasets/rohanrao/
formula-1-world-championship-1950-2020`

Extract the archive and place all `.csv` files directly in `data/raw/` (13 files: `circuits.csv`, `drivers.csv`, `races.csv`, `results.csv`, etc.). The dataset follows the Ergast API schema and is updated after each season.

10.2. Quick Start

With GraphDB running and the CSV files in place, a single script handles everything — virtual environment, dependencies, repository creation, RDF generation, server-side import, SPIN inference, and Django migration:

```
bash scripts/setup.sh
bash scripts/run_webserver.sh
```

Open <http://localhost:9000>. The admin panel is at <http://localhost:9000/admin-panel/login/>

Field	Value
Username	admin
Password	admin

These credentials are set by the `DJANGO_SUPERUSER_USERNAME` and `DJANGO_SUPERUSER_PASSWORD` variables in `.env` and created automatically by `setup.sh` on first run.

`setup.sh` skips steps that are already complete and can be re-run safely.

10.3. Manual Step-by-Step

Listing 37: Complete setup from a fresh clone

```
# 1. Virtual environment and dependencies
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. Environment configuration
cp .env.example .env
# Defaults work as-is. Add GEMINI_KEY for the F1 Assistant page.

# 3. Create GraphDB repository (owl-max-optimized ruleset -- required)
bash scripts/create_graphdb_repo.sh

# 4. Generate RDF facts from CSV (no rdf:type for base classes)
python3 scripts/csv_to_rdf.py # -> data/rdf/formula1.nt
(~798 MB)

# 5. Merge ontology + facts
python3 scripts/merge_integrated.py # -> data/rdf/
formula1_integrated.nt

# 6. Load into GraphDB (~2 min, server-side import)
bash scripts/load_rdf_to_graphdb.sh

# 7. Run all 21 SPIN inference rules (~1 min)
python3 manage.py run_spin_rules # expected: 21/21 rules
applied

# 8. Django setup (SQLite for auth only)
python3 manage.py migrate
python3 manage.py createsuperuser

# 9. Start the server (port 9000)
bash scripts/run_webserver.sh
```

Re-running after an ontology change: Delete `formula1_integrated.nt` and re-run from step 5. `setup.sh` detects the timestamp difference automatically:


```
rm data/rdf/formula1_integrated.nt
bash scripts/setup.sh
```

10.4. Protégé Validation

To validate the ontology in Protégé 5.6.9 with HermiT:

1. Open Protégé and load `data/rdf/formula1_integrated.nt` (File → Open from File, format: N-Triples).
2. Select Reasoner → HermiT 1.4.3.
3. Click Start Reasoner. HermiT completes in under 1 second on schema only (0 explicit individuals — all types are inferred from properties).
4. Navigate to the Entities tab → Classes to verify the inferred class hierarchy matches Section 2.9.
5. No unsatisfiable classes should appear (red background on any class would indicate a modelling error).

Note on `xsd:date` / `xsd:gYear`: The ontology intentionally omits `rdfs:range` declarations for `f1:dob`, `f1:date`, and `f1:year` because `xsd:date` and `xsd:gYear` are not in the OWL 2 datatype map (Section 2.9). The actual data values in GraphDB remain typed as `xsd:date/xsd:gYear` — only the ontology range assertions are omitted to maintain full OWL 2 DL compliance.

10.5. Re-running Inference

All SPIN rules are idempotent. To re-run after a data change:

```
python manage.py run_spin_rules
```

10.6. Data Files Summary

File	Purpose	In git?
<code>data/rdf/formula1_ontology.ttl</code>	OWL 2 DL ontology (hand-authored)	Yes
<code>data/rdf/formula1.nt</code>	Facts only, no <code>rdf:type</code> for base classes	No (generated)
<code>data/rdf/formula1_integrated.nt</code>	Ontology + facts merged (for Protégé)	No (generated)
<code>championship/spin_rules.py</code>	21 SPIN rules as Python module	Yes